



Christophe PICAUD
Architecte Microsoft chez
'Modern Applications by Devoteam'
christophepichaud@hotmail.com
www.windowscpp.com

Introduction à ASP.Net Core 3.0

ASP.Net Core est le framework multiplateformes et open source de Microsoft pour faire des applications Web. Cela peut être des sites web, des services Web API, des apps IoT et cela tourne sur le cloud ou sur des serveurs on-premise.

Choix de Framework

La table ci-dessous compare ASP.NET Core et ASP.NET 4.x.

ASP.NET Core	ASP.NET 4.x
Cible Windows, macOS, ou Linux	Cible Windows
Razor Pages est le modèle de développement recommandé pour créer des UI Web avec ASP.NET Core 3.0. Voir aussi MVC, Web API, and SignalR.	Utilise Web Forms, SignalR, MVC, Web API, WebHooks, or Web Pages
Plusieurs versions par machine	Une version par machine
Développement avec Visual Studio, Visual Studio for Mac, ou Visual Studio Code zn utilisant C# ou F#	Développement op with Visual Studio using C#, VB, or F#
Meilleure performance que ASP.NET 4.x	Bonne performance
.NET Framework ou .NET Core runtime	Utilise le .NET Framework runtime

Pourquoi ASP.NET Core?

Asp.Net 4.x est un framework très utilisé. ASP.NET Core est la nouvelle version, plus rapide, plus modulaire. On y trouve les fonctionnalités suivantes :

- Une API unifiée pour les UI web et les web APIs ;
- Architecturé pour la testabilité ;
- Les pages Razor facilitent et rendent plus productifs les scénarios axés sur la page de codage ;
- Possibilité de développer et d'exécuter sur Windows, macOS et Linux ;
- Open source et axé sur la communauté ;
- Intégration des frameworks modernes, côté client et des workflows de développement ;
- Un système de configuration basé sur l'environnement, prêt pour le Cloud ;
- Injection de dépendances intégrée ;
- Un pipeline de requêtes HTTP léger, performant et modulaire ;
- Possibilité d'héberger sur IIS, Nginx, Apache, docker ou auto-Host dans votre propre processus ;
- Versionning des applications côte à côte avec .NET Core ;
- Outillage qui simplifie le développement Web moderne.

Fonctionnalités .NET Core 3 dans les diverses Preview

Pour vous donner une idée de la dynamisme du projet ASP.NET Core, voici la liste des fonctionnalités proposées dans les diverses Preview de .NET Core 3.

.NET Core 3.0 Preview 2 propose :

- Razor Components ;
- Le streaming SignalR client-server ;
- Les Pipes sur HttpContext ;
- Des templates de host génériques ;
- Des mise à jour sur les routing Endpoint.

.NET Core 3.0 Preview 3 propose :

- Améliorations des composants Razor :
 - Template de projet simple,
 - Nouvelle extension.razor,
 - Intégration des routing Endpoint,
 - Prerendering,
 - Composants Razor dans des DLL de classes Razor,
 - Gestion des événements améliorée,
 - Forms & validation.
- Compilation au Runtime ;
- Template de Worker Service ;
- Template gRPC ;
- Template Angular mis à jour pour Angular 7 ;
- Authentification pour les Single Page Applications ;
- Intégration SignalR avec les routing endpoints ;
- Support SignalR Java client pour le polling long.

.NET Core 3.0 Preview 4 propose :

- Composants Razor renommés Blazor server ;
- Blazor client sur WebAssembly est officiel ;
- Resolution de nom de composants basée sur `@using` ;
- `_Imports.razor` ;
- Template Nouveau composant ;
- Reconnexion au même serveur ;
- Reconnexion stateful après prerendering ;
- Composants interactifs de rendu stateful depuis les pages et vues Razor ;
- Détection quand une app fait du prerendering ;
- Configuration du client SignalR pour les apps Blazor server ;
- Amélioration des fonctionnalités de reconnexion SignalR ;
- Configuration du client SignalR pour les apps Blazor server ;
- Options additionnelles pour le service de registration MVC ;
- Mises à jour des routing Endpoint ;
- Nouveau template gRPC ;
- Design-time build pour gRPC ;
- SDK New Worker.

Réaliser des application web UI et des web APIs avec ASP.NET Core MVC

ASP.NET Core MVC fournit des fonctionnalités pour créer des API Web et des applications Web :

- Le modèle MVC (Model-View-Controller) rend vos API Web et vos applications Web testables ;
- Les pages Razor sont un modèle de programmation basé sur une page qui rend l'UI Web plus facile à réaliser et plus productive ;
- Les balises Razor sont une syntaxe productive pour les pages Razor et les vues MVC ;
- La prise en charge intégrée de plusieurs formats de données et la négociation de contenu permet à vos API Web d'atteindre un large éventail de clients, y compris les navigateurs et les appareils mobiles ;
- Le modèle de binding mappe automatiquement les données des requêtes HTTP aux paramètres de la méthode d'action ;
- Le modèle de validation effectue automatiquement la validation côté client et côté serveur.

Développement client

ASP.NET Core intègre facilement les frameworks clients (JS, TS) les plus connus, comme Blazor, Angular, React, and Bootstrap.

ASP.NET Core et .NET Framework

ASP.NET Core 2.x peut tourner sur .NET Core ou .NET Framework. Les applications ASP.NET Core pour .NET Framework ne sont pas cross-platforms - elles ne tournent que sur Windows.

ASP.NET Core 3 ne tourne que sur .NET Core. Les avantages de cibler .NET Core sont :

- Cross-platform. Tourne sur macOS, Linux, et Windows ;
- Meilleures performances ;
- Exécution Side-by-side ;
- Nouvelles APIs ;
- Open source.

Introduction à ASP.NET Core MVC

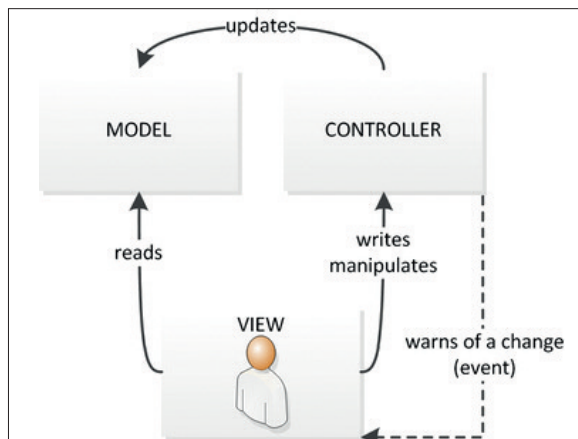
ASP.NET Core MVC est un Framework riche pour la création d'applications Web et d'API à l'aide du pattern Model-View-Controller.

Le pattern MVC

Le pattern d'architecture Model-View-Controller (MVC) architectural sépare une application en 3 groupes de composants : Models, Views, et Controllers. À l'aide de ce modèle, les demandes des utilisateurs sont acheminées vers un contrôleur qui est responsable de l'utilisation du modèle pour effectuer des actions utilisateur et/ou récupérer les résultats des requêtes. Le Controller choisit la vue à afficher à l'utilisateur, et lui fournit toutes les données de modèle qu'il exige. **1**

Responsabilités du modèle

Le Model dans une application MVC représente l'état de l'application et toute logique métier ou opérations qui doivent être exécutées par elle. La logique métier doit être encapsulée dans le modèle, ainsi que toute logique d'implémentation pour la persistance de l'état de l'application. Les vues fortement typées utilisent généralement des types ViewModel conçus pour contenir les données à afficher sur cette vue. Le contrôleur crée et remplit ces instances ViewModel à partir du modèle.



Responsabilités des vues

Les vues sont chargées de présenter le contenu via l'interface utilisateur. Elles utilisent le moteur de vue Razor pour incorporer du code .NET dans le balisage HTML. Il devrait y avoir une logique minimale dans les vues, et toute logique en elles doit se rapporter à la présentation de contenu. Si vous trouvez la nécessité d'effectuer une grande partie de la logique dans les fichiers de vue afin d'afficher des données à partir d'un modèle complexe, envisagez d'utiliser un composant de vue, ViewModel ou un modèle de vue pour simplifier la vue.

Responsabilités du contrôleur

Les contrôleurs sont les composants qui gèrent l'interaction de l'utilisateur, travaillent avec le modèle et sélectionnent finalement une vue à restituer. Dans une application MVC, la vue affiche uniquement les informations; le contrôleur gère et répond à l'entrée et à l'interaction de l'utilisateur. Dans le modèle MVC, le contrôleur est le point d'entrée initial. Il est responsable de la sélection des types de modèles à utiliser et de la vue à restituer (d'où son nom). Il contrôle la façon dont l'application répond à une requête.

ASP.NET Core MVC

ASP.NET Core MVC est un Framework léger de présentation, open source, hautement testable, optimisé pour une utilisation avec ASP.NET Core. ASP.NET Core MVC fournit un moyen basé sur des modèles pour créer des sites Web dynamiques qui permet une séparation nette des responsabilités. Il vous donne un contrôle total au travers des balises, prend en charge le développement compatible TDD et utilise les dernières normes Web.

ASP.NET Core MVC fournit les services suivants :

- Routing ;
- Model binding ;
- Model validation ;
- Dependency injection ;
- Filters ;
- Areas ;
- Web APIs ;
- Testability ;
- Razor view engine ;
- Strongly typed views ;
- Tag Helpers ;
- View Components.

Architecture ASP.NET Core 2

Hébergement : processus natif

Au bas de la figure est la couche d'hébergement. L'hébergement est une petite couche de code natif qui est responsable de trouver et appeler l'hôte natif. Plusieurs hôtes sont disponibles, dont le [ASP.NET Core Module](#), Kestrel et DOTNET. exe. Le [ASP.NET Core module](#) permet aux applications [ASP.NET Core](#) d'être hébergées dans IIS. Dotnet. exe est un outil de ligne de commande qui peut être utilisé pour créer et exécuter vos applications [ASP.NET](#) à des fins de développement et de test. Kestrel est un serveur HTTP multiplateforme haute performance basé sur la bibliothèque d'e/s asynchrones libuv. Il offre les meilleures performances pour héberger votre application [ASP.NET Core](#). Cependant, Kestrel n'offre pas le même niveau de fonctionnalités comme IIS. Par exemple, si votre application doit utiliser l'authentification Windows, vous devez utiliser Kestrel en conjonction avec IIS.

Si vous en avez besoin, vous pouvez créer un hôte personnalisé pour héberger votre application ASP.NET. Cela pourrait être une Application WPF, un outil de ligne de commande ou un service Windows.

Runtime: CLR Native Host

La couche Runtime configure et démarre le CLR et crée le domaine d'application pour le code managé pour tourner dedans. Lorsque l'application d'hébergement s'arrête, la couche d'exécution est chargée de nettoyer les ressources utilisées par le CLR, puis l'arrêter. Dans la couche Runtime, les machines Windows auront une implémentation de code natif du CLR. Vous avez également la possibilité d'exécuter l'implémentation mono du CLR. Le projet mono offre des hôtes natifs pour macOS, Linux et Windows.

Runtime : point d'entrée managé

Le point d'entrée managé est écrit en code natif. Le but principal de cette couche est de trouver et de charger les assemblages requis. Une fois les assemblages chargés, le point d'entrée managé est appelé et l'application commence à s'exécuter.

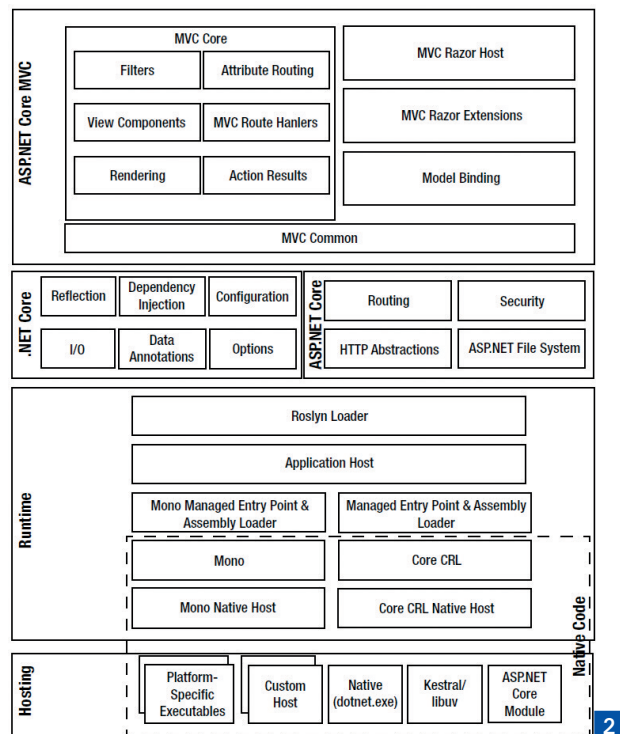
Runtime : hôte d'application

La couche d'hébergement d'application du runtime est la première couche dans laquelle un développeur Web est généralement impliqué. L'hôte de l'application lit le fichier .csproj du projet et détermine les dépendances. Il peut localiser des assemblages à partir de nombreuses sources, y compris NuGet et les assemblages compilés en mémoire par les services du compilateur Roslyn. Dans une application [ASP.NET Core](#), cette couche créera également le pipeline [ASP.NET Core](#) et chargera les composants middleware spécifiés.

Runtime : Roslyn Loader

Le chargeur Roslyn est responsable du chargement et de la compilation des fichiers sources. Avec [ASP.NET Core](#), le code de l'application n'a pas besoin d'être compilé avant son déploiement. Il peut être déployé en tant que code source et compilé à la demande.

Dans la pile [ASP.NET Core](#), il existe de nombreux composants requis par [ASP.NET Core MVC](#). Certains des plus importants sont indiqués dans le schéma ci-dessus.



- **Routing** : ce composant contient la logique permettant de mapper les requêtes HTTP composant d'application ou de ressource statique. Il vous permet de personnaliser l'URL pour votre application ou service Web.
- **Sécurité** : ce composant est constitué d'une collection de fournisseurs basés sur OWIN connus collectivement comme [ASP.NET Identity](#). [ASP.NET Identity](#) prend en charge plusieurs authentifications et les normes d'autorisation, y compris l'authentification SAML, OAuth et Windows, ainsi que des bases de données utilisateur custom avec l'authentification des cookies. Cette fonctionnalité vous permet d'intégrer rapidement votre application à des fournisseurs d'identité externes tels que Facebook, Microsoft et Google ou d'intégrer votre application à Active Directory. [ASP.NET Identity](#) inclut également la prise en charge des fonctionnalités de sécurité avancées comme l'authentification à deux facteurs.
- **Abstractions http** : ce sont de nouveaux composants qui font partie d' [ASP.NET Core](#). Ils créent une couche d'abstraction qui garantit des API et des comportements cohérents, quel que soit l'endroit où vous hébergez votre application [ASP.NET Core](#).
- **Système de fichiers [ASP.NET Core](#)** : cela fournit la gestion des fichiers statiques pour vos applications Web. Ceci est nécessaire car contrairement aux versions antérieures de ASP.NET, qui ont délégué le chargement et la portion de fichiers statiques tels que des images et des fichiers CSS sur le serveur Web, [ASP.NET Core](#) avait besoin d'un ensemble standard d'abstractions qui seraient cohérentes entre les différents serveurs Web et systèmes d'exploitation.

Créer une application ASP.NET Core avec Visual Studio 2019

Les nouveaux développements doivent se faire avec des pages Razor. C'est la préconisation Microsoft. Lançons Visual Studio 2019 et nous allons créer une nouvelle Application Web ASP.NET Core.

3 4

Nous choisissons une Web Application non MVC. 5

Voici le contenu du Solution Explorer : 6

Le projet cible .NET Core 3 preview 4. On remarque le folder Pages et les pages Razor... C'est très minimaliste.

Une page Razor

Une page Razor c'est :

- Un fichier cshtml en HTML avec des balises Razor @xxx ;
- Un fichier cshtml.cs avec du code en C#.

Voici le contenu de la page index.cshtml :

```
@page
@model IndexModel
@{
    ViewData["Title"] = "Home page";
}

<div class="text-center">
    <h1 class="display-4">Welcome</h1>
    <p>Learn about <a href="https://docs.microsoft.com/aspnet/core">building Web apps
    with ASP.NET Core</a>.</p>
</div>
```

Voici le contenu du fichier index.cshtml.cs :

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
```

```
namespace MyApp2.Pages
{
    public class IndexModel : PageModel
    {
        public void OnGet()
        {
        }
    }
}
```

Conclusion

Le Framework ASP.NET Core est riche et performant. C'est une composante essentielle à connaître pour tout développeur .NET. C'est le point d'entrée vers les architectures micro-services. Nous le verrons dans le prochain dossier...

